

dr. Matej Šprogar

Nasveti za programiranje

Ne ponavljajmo napak programerjev pred nami...



Izkušnja

Ne pozabimo, da je programski jezik zgolj orodje. In kot vsako orodje ga lahko uporabimo narobe.

Učimo se iz dobrih in slabih izkušenj: česa "ne smemo" in kaj "moramo" (je pametno).

Slabo je, če ... (vrstni red *ni* pomemben)

- slabo poimenujemo
- uporabljamo *goto*
- uporabljamo *globalne* spremenljivke
- pišemo "božje" funkcije
- pišemo specializirano kodo
- podvajamo kodo in/ali podatke
- pišemo krhko kodo
- uporabljamo programerske trike
- uporabljamo nekaj, ker je "moderno"
- ne razumemo tehnologije, ki jo uporabljamo
- pišemo grdo kodo
- dokumentiramo očitno namesto neočitnega
- izumljamo toplo vodo
- prehitro optimiziramo
- ...

ZA VEČ NASVETOV MORAMO ŠE BOLJE SPOZNATI PROGRAMSKE JEZIKE IN TEHNOLOGIJE

Slaba imena so grozna stvar!

Z dobrim poimenovanjem funkcij in pametno izbiro argumentov moramo doseči, da bo naša koda lahko razumljiva in da jo bo težko uporabiti narobe...

Neizrazna imena lahko popolnoma skrijejo namen programa in tudi morebitno napako. Enako nevarna so tudi predolga imena, ker jemljejo fokus kodi.

Napačna imena pa sploh spremenijo dožemanje programa (primer na desni)

```
// vaxner programer
void cepljenje(SeznamOseb pametni_ljudje) {
    for (Oseba x : pametni_ljudje)      // Java
        x.cepi(new VarnoCepivo());
}

// anti-vaxner programer
void eksperiment(SeznamOseb ovce) {
    foreach (Oseba x in ovce)           // C#
        x.cepi(new NепreverjenoCepivo());
}
```

Poimenovanje **izredno** vpliva na naše razumevanje problema, oblikuje naš način razmišljanja in posledično ustvarja program.

besede imajo moč!

Modernejši primer: ChatGPT "*razume*"!?

Slab je ... goto

Namesto ukaza *goto* imamo programske strukture, ki izvedejo ta isti ukaz na kontroliran način na kratki razdalji in z malo možnostjo napake.

Včasih pa je *goto* elegantnejša rešitev. Takrat pazimo, da ga izvedemo na čim krajši "razdalji" (po možnosti v okviru istega bloka kode) in dobro poimenujemo lokacijo skoka.

Za skok naprej uporabimo *if* stavek.

Za skok nazaj imamo zanke.

Za izstop iz zanke uporabimo *break*.

Za zaključek funkcije uporabimo *return*.

Za signalizacijo napake na višji nivo uporabimo *throw*.

Šele če to ne zadostuje, razmislimo o *goto*...

Slabe so globalne spremenljivke

Uporaba spremenljivk, ki so definirane izven konteksta trenutne funkcije, veča možnost zahrbtnih napak, tipično ravno v moderni, večnitni kodi.

Globalni podatek NI v ekskluzivni lasti funkcije in se lahko **nepričakovano** spremeni tekom izvajanja funkcije!

Dejansko so tudi razredni atributi "globalne" spremenljivke na ravni objekta, statični pa celega razreda!

Dobro je razumeti prednosti funkcijskega* programiranja (FP)!

```
void prva()
{
    if (global == 42) {
        bla_bla;           // Thread.Sleep(1)
        izpiši(global);    // WTF 7!?!
    }
}

void druga()
{
    bla_bla_bla;          // Thread.Sleep(1)
    global = 7;
}

int global = 42;
void main()
{
    zaženi_vzporedno(prva, druga);
}
```

Ni vse, kar je lokalno, tudi varno

Če ima naša funkcija **svoje** podatke, lahko dela z njimi kar hoče, pa ne bo vplivala na druge... Lokalni podatki so v toliko varni, nevarnost pa preti pri uporabi referenc!

Nevarno (in zakomplicirano) je, če imamo posreden dostop do stvari, ki jih ne rabimo, saj obstaja možnost napačne uporabe.

Če uporabljamo *skupne* vsebine se zavedajmo nevarnosti in po možnosti takšne podatke le **berimo**. Pri pisanju vedno preverimo, ali bo kdo prizadet ob spremembi podatka!

// SLABO

```
static void poslji(Oseba oseba, string text)
{
    poslji(oseba.naslov, text); // 😞 KOMPLICIRANO!
    oseba.naslov = nek_naslov;  // 😱 LEGALNO!?
```

// DOBRO

```
static void poslji(Naslov naslov, string text)
{
    // ne moremo "pokvariti" osebe, ker je ni
    // vse kar rabimo, je na pladnju
    // vidimo samo potrebno, nič več in nič manj
}

poslji(Bozicek, pismo);           // uh?
poslji(Bozicek.naslov, pismo);    // varno
```

Slaba je neizrazna koda...

Če ukaz v kodi ne odraža vsega, kar se bo zgodilo, je nekaj narobe. Lahko so krive globalne spremenljivke ali slabo poimenovanje, ponavadi pa slaba zasnova programa.

Skušajmo pisati funkcije, ki so osredotočene na eno nalogo in dajo en rezultat, **vse potrebno** pa dobijo preko svojih argumentov. V tem primeru bo klic *vedno* odražal vse, kar je pomembno.

V nasprotnem primeru nastane "magična" koda, kjer so podrobnosti skrite, mnogo stvari pa se dogaja v ozadju, **mimo** bralca kode!

Kodo, ki določi ceno izdelka, lahko napišemo na več načinov, recimo:

1. `določi_ceno(izdelek);`
2. `cena = določi_ceno(izdelek);`
3. `cena = določi_ceno(izdelek, popust);`

Čeprav bodo delale vse variante, prvi dve rešitvi ne odraža vsega, kar je pomembno; zadnji klic pove mnogo več in v toliko ne potrebuje dodatnih komentarjev.

Slabe so "božje" funkcije (in objekti)

Pogost problem so funkcije, ki delajo **preveč** stvari (vse opravijo same).

Funkcije uporabljamo za razgradnjo problema in idealna funkcija ima eno odgovornost (naloge), obsega pa največ 1 ekran (<50 vrstic) kode.

Večje stvari moramo doseči z dekompozicijo na več podfunkcij, ki jih spet razbijamo... vse do programskih konstruktov nad našimi tipi.

Če ima funkcija (pre)več zadolžitev, je ne moremo ponovno uporabiti kje drugje, ima večjo možnost napake in je težje preverljiva.

Velike funkcije nič nalog ne prepuščajo drugim funkcijam, najraje preskočijo več nivojev poveljevanja, so dolge in zapletene za branje... Ne uporabljajo principa *deli in vladaj*.

```
void oponašaj_psa() {  
    ...  
    if (stanje == Razpoloženje.Vesel) {  
        rep.položaj = i%2 ? +1 : -1;  
        jezik.glas += 1;  
    }  
    else if (stanje == Razpoloženje.Lačen)  
    ...  
    ...  
}
```

Podobno tudi božji objekti zmorejo VSE sami.

Slaba je specializirana koda

Specializirana koda je uporabna samo v posebnih primerih in je v toliko ne moremo uporabiti drugod. Dodatno jo je težko spreminjati, popravljati in sestavljati z drugo kodo.

Znak specializirane kode so dolga imena funkcij - splošna funkcija ima kratko ime, ki odraža njen namen, ime specializirane funkcije mora odražati tudi specializacijo...

Nasprotje temu je *splošna (generična)* koda, ki pokriva čim več primerov uporabe; o tem več drugje.

Specializirane funkcije so pogosto odvisne od globalnih spremenljivk in konstant, klicajoča koda lahko hitro kliče napačno specializacijo...

```
void oponašaj-veselo-činčilo() {  
    if (teža < 10)  
        cvili();  
    else  
        skači();  
}
```

Posledično rabimo funkcije za vse pasme vseh razpoloženj... Nastane neobvladljiva množica podobno imenovanih funkcij.

Pogosta rešitev so virtualne* funkcije:

```
void oponašaj-veselega_psa(Pes pes) { pes.vesel(); }
```

Slabo je podvajanje kode

Podvajanje kode pomeni, da bo pri morebitnih spremembah potrebno popravljati kodo **na več mestih**. To poveča potrebno količino dela in predvsem tudi možnost napak.

Podvajanju se v osnovi izogibamo z zankami (zaporedno ponavljanje), funkcijami (večkratno ponavljanje blokov kode) in šablonami/generiki* (tipsko ponavljanje kode).

Najbolje je uporabiti že napisano kodo (knjižnice, stari projekti...)

Ponavljjanje pomeni daljšo kodo, ki kar kliče napake zaradi površnosti. Če že imamo neko kodo, jo je neumno ne uporabiti znova...

```
for (int i=0; i<tab1.Length; ++i)
    sum1 += tab1[i];
avg1 = sum1 / tab.Length;
for (int i=0; i<tab2.Length; ++i)
    sum2 += tab2[i];
avg2 = sum2 / tab.Length;
```

je bolje zapisati recimo kot:

```
avg1 = povprecje(tab1);
avg2 = povprecje(tab2);
```

Najbolje je:

```
avg1 = tab1.Average();           // C#
avg2 = Arrays.stream(tab2).average(); // Java
```

Slabo je podvajanje podatkov

Ohranjanje iste vrednosti na več lokacijah je izredno nevarno, saj zahteva popolno **sinhronizacijo** podatkov - če se vrednosti razlikujejo in ne vemo zakaj, potem tudi ne vemo, katera je pravilna!

Kopiranje funkcijskih argumentov ne pomeni nevarnega podvajanja podatkov, saj imajo te kopije kratko življenjsko dobo.

Problem kaže podobne simptome kot problem globalne spremenljivke.

```
podatek = DB.beri_podatek();  
kopija = podatek;  
  
...  
spremeni(podatek);           // kaj pa kopija?  
  
...  
uporabi(kopija);             // kopija != podatek
```

Vsak objekt je pravzaprav kopija realnosti (original). Če smo edini uporabnik in imamo en objekt ni težav, če pa dela z originalom še kdo drug ali pa si sami naredimo več trajnih kopij... imamo **VELIKO** težavo.

Primer smiselnega podvajanja podatkov je predpomnilnik, ki pa zahteva več kot samo izrednega programerja...

Slaba je krhka koda

S pridevnikoma "krhko" in "robustno" označujemo stopnjo stabilnosti kode ob spremembah. Krhko kodo poruši že majhna sprememba v smislu, da povzroči veliko sintaktičnih napak. Robustna koda lahko nasprotno tolerira tudi večje posege brez težav.

Krhka koda se tipično pokaže, ko moramo v obstoječem programu spremeniti neko funkcionalnost...

Robustnost zahteva zelo premišljeno zasnovo kode (funkcij, razredov, ...) in jo je težko doseči "za nazaj".

Slabo:

```
int alfa = 0;

int delaj() {
    return alfa + 1;
}

int main() {
    alfa = delaj();
}
```

Npr. preimenovanje spremenljivke *alfa* bo zrušilo VSE funkcije, ki jo uporabljajo...

Bolje:

```
int alfa = 0;

int delaj(int alfa) {
    return alfa + 1;
}

int main() {
    alfa = delaj(alfa);
}
```

Funkcija *delaj* je sedaj neodvisna od imena zunanje spremenljivke *alfa*.

Slabi so triki

Cilj programiranja ni dokazati nesrečnim programerjem, ki bodo brali našo kodo, da smo pametnejši od njih.

Samo *najboljši* programerji zmorejo napisati *preprosto* kodo, kompliciranje je znak neznanja.

Vsak (pol)trik moramo izčrpno dokumentirati.

Koda naj bo očitna, sicer bomo tudi sami imeli težave čez eno leto, ko jo bomo skušali (ponovno) razumeti.

Kaj naredi naslednja koda?

```
int x, y;
```

1. $(x \wedge y) < 0;$ // $x * y < 0$
2. $y \wedge ((x \wedge y) \ \& \ -(x < y));$ // $\min(x, y)$
3. $x \wedge= y;$ // $x \leftrightarrow y$
 $y \wedge= x;$
 $x \wedge= y;$
4. ...

Določeni triki so upravičeni le, če to brezpogojno zahteva optimizacija kode (morebitne strojne in prostorske omejitve).

Moderno *ni vedno* tudi dobro...

Objektno programiranje je bilo v 80. in 90. letih zakon; kar ni bilo objektno, je bilo slabo. Danes vemo, da ni tako.

Industrija nam vsak dan skuša vsiliti neko tehnologijo kot zakon in vse neuporabnike slika kot nazadnjaške...

Edini kriterij je *dolgoživost* "prodajanih" izdelkov (prgm. jeziki, sistemi, platforme, ogrodja, aplikacije...)

There's no free lunch!

Kdo se spomni/pozna:

Windows Phone, Windows Vista, Java OS,
Google Plus, Silverlight, SOAP, ...

Velike vojne še trajajo, nastajajo nove:

Win|Linux, Java|C#, FOSS/©, ...

Je pa res, da zmagovalci niso nujno tudi najboljša izbira, ampak stvar marketinga in trenutka:

VHS/Beta, x86/m68k, Java, PHP, JS...

Recimo, če jezik ponuja dedovanje to še ne pomeni, da ga *moramo* uporabiti. Ali vmesnike. Ali karkoli. Če ne razumemo tehnologije 100%, jo uporabljajmo skrajno konzervativno!

Slabo je, če ne razumemo tehnologije, ki jo uporabljamo

Za nek problem obstaja mnogo rešitev in kriterij, da program deluje, še zdaleč ni dovolj. Prepogosto lahko pridemo do delujočih rešitev zgolj s slepim "klikanjem" ali ugibanjem...

Hitra rešitev se dolgoročno ponavadi sprevrže v nočno moro, ko je treba odpravljati posledice.

Vsako orodje ima svoj namen, čeprav se kladivu vse zdi žebelj!

Uporaba objektov sama po sebi ne pomeni, da je koda boljša, varnejša. Nasprotno je pogosto nerazumevanje, kaj objekti zmorejo in česa ne, privedlo do njihove napačne uporabe in posledično kritiziranja. Kladivo ni krivo, če razbije vazo...

Tudi funkcijsko programiranje ima svojo temno plat, ki jo pravzaprav naslavlja OOP.

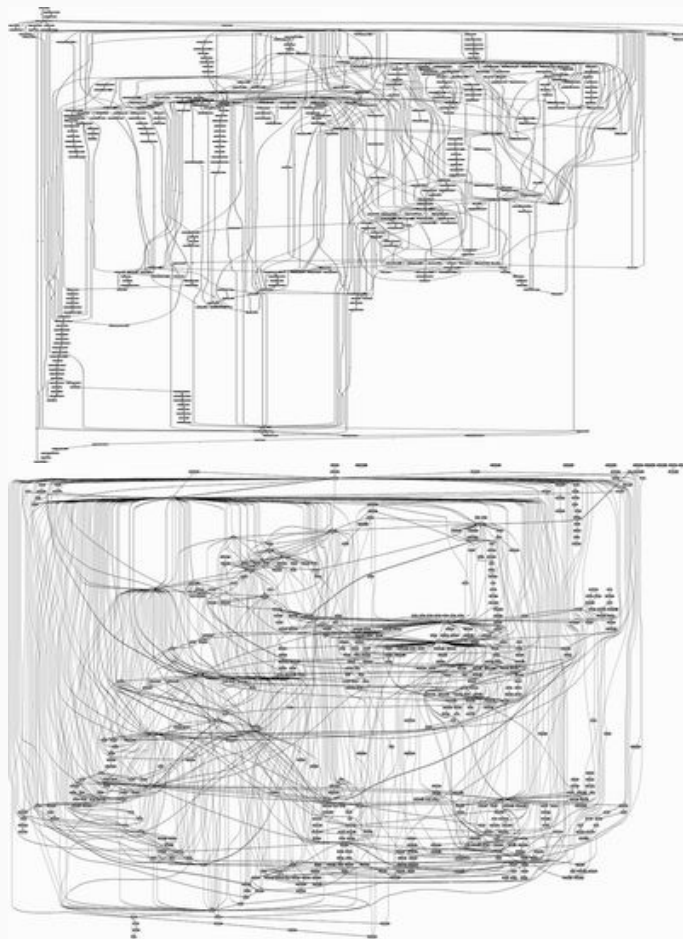
Optimum je, kot vedno, kombinacija obeh pristopov.

"Grda" koda je slaba koda

Vsaj lepopis je z modernimi IDE enostaven, saj nudijo avtomatsko oblikovanje. Še vedno pa lahko berljivost pokvarimo z nepotrebniimi ukazi, slabo strukturo, prezapletenimi odvisnostmi, ...

Najgrša je slabo zasnovana koda, ki dopušča več izvajalnih poti, kot jih lahko programer obvlada.

Primer na desni so vizualizirani sistemski klici v Apache strežniku (zgoraj) in ista naloga v Microsoftovem IIS (spodaj); pisalo se je leto 2006.



Slabo je ponavljati že napisano

Programski jezik je namenjen usposobljenemu človeškemu bralcu, torej ni treba ponavljati očitnega. Še huje, takšno ponavljanje je škodljivo, ker zasiči vidno polje bralca in prikrije resnično pomembne informacije!

Rešitev je dobro poimenovanje in dokumentiranje NEOČITNIH stvari, ki jih bralec rabi za razumevanje kode.

Avtomatsko generiranje dokumentacije je pogosto žal del problema in ne rešitve, čeprav je namen dober.

```
/* Določi stanje glede na moč luči
 * Sprejme en parameter - luč
 * vrne 0=ugasnjeno, 1=zatemnjeno, 2=vklopljeno */
public int vrni_stanje(int moc_luci)
{
    if (moc_luci == 0)           // ni porabe
        return 0;                // ugasnjeno
    if (moc_luci != 100)         // polna moc je 100
        return 1;                // zatemnjeno
    return 2;                    // prizgano
}

public Stanje vrni_stanje(Svetilo luc)
{
    if (luc.moc == Svetilo.NiPorabe)
        return Stanje.Ugasnjeno;
    if (luc.moc != Svetilo.PolnaMoc)
        return Stanje.Zatemnjeno;
    return Stanje.Prizgano;
}
```

Slabo je, če je projekt nedokumentiran...

Gola koda le redko popolnoma opiše svoj namen in delovanje, zato moramo *dodatno* dokumentirati vse, kar iz kode ni razvidno.

Ločeno moramo dokumentirati tudi način uporabe, arhitekturne odločitve, arhivirati je potrebno vse razvojne stopnje projekta s spremembami, korake zagotavljanja kakovosti, projektne odločitve...

Podporne tehnologije (npr. *git*) boste spoznali drugod...

```
/*
```

```
Točkovanje objekta je vmesni korak pri cenitvi  
zemljišča po scenariju A ali B, ne pa tudi C
```

```
*/
```

```
private int tockujAB(Parcela parcela, Pravilnik P)  
{  
  
    predpogoj(velikost() <= parcela.velikost());  
    predpogoj(etaze() <= P.MaxSteviloEtaz);  
  
    int točke = 0;  
    ...  
    // posebna zahteva GURS 31.10.2021  
    tocke += P.točkujRabo(parcela.tip_rabe());  
    ...  
    popogoj(tocke >= 0 && tocke <= P.MaxTocke);  
    return tocke;  
}
```

Slabo je izumljati toplo vodo

Ogromno stvari je že narejenih in neumno je, da jih ne uporabimo. To zahteva vsaj **poznavanje** knjižnic, ki jih imamo na razpolago.

Temeljne stvari niso enostavne, če jih želimo implementirati optimalno.

Programirajmo tisto, kar še ni narejeno. Hkrati uporabimo čim več že napisane in preizkušene kode, tudi če se nam zdi, da bi znali bolje...

Primer sortiranja:

Naivna izvedba sortirnega algoritma ima zahtevnost $O(n^2)$ in že pri manjši količini podatkov porabi veliko časa.

Izvedba dobrega algoritma ima zahtevnost $O(n \cdot \log n)$, a še vedno deluje občutno počasneje kot ta ISTI algoritem v optimizirani obliki v programski knjižnici.

Enako velja za vse druge algoritme, podatkovne strukture, ogrodja, ...

Slabo je, če kodo optimiziramo že med pisanjem

Pravilno delovanje moramo doseči z berljivo/razumljivo kodo, šele nato se osredotočimo na učinkovitost.

Prehitro razmišljanje o učinkovitosti kode pogosto pomeni, da se programer zaplete in izgubi v podrobnostih in posledično sploh ne pride do cilja.

Večinoma je bolje najprej priti do cilja in nato za nazaj optimizirati kodo.

Kodo optimizirajmo šele na podlagi konkretnih realnih meritev (*profiler*).

Pogosto se izkaže, da program med izvajanjem največ časa ne izgubi v nekih detajlih, za katere smo *mislili*, da so pomembni, ampak nekje čisto drugje. Fokus optimizacije je tako znan šele, ko program deluje in ga lahko opazujemo in merimo!

Le če imamo stroge časovne in prostorske omejitve, jih moramo imeti v mislih na ravni načrtovanja programa; v nasprotnem primeru jih mogoče z optimizacijo slabe kode sploh ne bomo dosegli.

Napake so ... nezaželene.

Koda naj dopušča čim manj napak - funkcije skušajmo zasnovati tako, da do napak ne pride oziroma je koda varna pred posledicami napak.

Izjeme so mehanizem za obravnavo napak in niso rešitev, zgolj orodje za izdelavo rešitve. Uporabljajmo jih izključno za obravnavo napak, čeprav omogočajo še kaj drugega.

Uporabljajmo vse: predpogoje, popogoje, izjeme in, če moramo, tudi tradicionalno obravnavo napak.

Primer: Prevajalnik lahko garantira, da bo funkcija prejela eno število, ne more pa garantirati, da bo v njem pravilna vrednost.

Prevajalnik tudi ne more garantirati, da bo poslana referenca kazala na veljaven/pravi objekt ali da bo ta objekt izpolnil naloge v skladu s pričakovanji...

Če gre karkoli narobe, mora koda to zaznati in odreagirati pravilno!

Dobro je ...

če ne delamo slabih stvari 😊

zahtevnejši nasveti v drugem semestru...